# ELI — The Built-in Code Editor

**ELI v2.5.35**

November 2026

## Contents


**ELI's built-in code editor (IDE) — a plain-English guide** **1**
- What it is . . . 1
- Opening it . . . 1
- The part that matters if you don't code: **let ELI do it** . . . 2
- The part if you *do* code . . . 2
- Making things that run — plots, graphs, and simulations . . . 2
- Quick reference . . . 2


## ELI's built-in code editor (IDE) — a plain-English guide

ELI has a **code editor built in** — you don't need to install VS Code or anything else. This guide explains what it does and how to use it, whether or not you write code yourself. Everything runs on your own machine.

---

### What it is

A clean, lightweight editor for looking at and changing code and text files, with:

- **Line numbers** down the side,
- the **current line highlighted** so you don't lose your place,
- **colour-coded Python** (keywords, strings, comments in different colours) so code is easy to read,
- **auto-indent** — it lines things up for you as you type.

It's built right into ELI (no extra install, no separate license), so it opens instantly and works offline.

---

### Opening it

- **Just ask ELI:** say or type *"open the IDE"* (or *"open the code editor"*).
- It opens as a panel inside ELI — no juggling separate windows.

---

## The part that matters if you don't code: let ELI do it

You don't have to write anything yourself. ELI has a **coding assistant** that plans, writes, runs, tests, and fixes code for you. Just describe what you want:

- **Write something new:** *"write me a script that renames all the photos in a folder by date"* → ELI plans it, writes it, runs it, and fixes it if it breaks.
- **Fix a file:** *"there's a bug in this file"* or *"fix errors in report.py"* → ELI scans it, shows you what it found, and repairs it **after you confirm**.
- **Check code for problems:** *"examine my_script.py for errors"* → ELI does a layered check (syntax → style → deeper analysis) and reports what's wrong.
- **Understand code:** *"what does this file do?"* → ELI reads it and explains in plain English.

ELI's coding assistant doesn't just guess — it **runs the code and tests it** before telling you it's done, and it remembers past bug→fix pairs so it gets better over time. It will always show you a change and wait for your OK before touching a file.

---

## The part if you *do* code

- Open a file, edit it in the editor, save it.
- Ask ELI to review or extend what you've written: *"add error handling to this"*, *"make this faster"*, *"write tests for this function"*.
- ELI can work across your whole project, not just one file.

---

## Making things that run — plots, graphs, and simulations

The editor pairs with ELI's scientific tools, so you can ask for things that actually **produce a result**, not just code:

- **2D:** *"plot this data as a line chart"*, *"make a bar graph of these numbers"*.
- **3D:** *"show a 3D surface of z = sin(x)·cos(y)"*, *"render this STL mesh and tell me its dimensions"*.
- **4D (things that change over time):** *"animate a bouncing ball"*, *"simulate a pendulum and show it moving"*, *"model how this population grows over 50 steps"*.

ELI writes the maths/simulation code, runs it, and shows you the plot, graph, or animation. (See `blueprints/simulations_and_plots.md` for the full range.)

---

## Quick reference

| You want to… | Say this |
|---|---|
| Open the editor | "open the IDE" |
| Have ELI write something | "write me a script that …" |
| Fix a file | "fix errors in " |
| Check for problems | "examine for errors" |
| Understand code | "what does do?" |
| Make a plot / simulation | "plot …" / "simulate …" / "animate …" |

You never have to leave ELI, install anything, or know what any of the code means — just describe what you want.